

Building a Two-Tier Financial Contract Auditor

How I paired an RL-trained Qwen2.5-7B agent with GLiNER for fast, deterministic contract extraction

Author: Kushagra Singh Kushwah | Project: llm-financial-analyst-agent | Technical Report

1. The Problem with Using Only a 7B LLM for Contract Review

In this project, my goal was to build an automated financial auditor that can review commercial contracts, flag late penalties, and calculate financial exposure. My initial approach was centered entirely around an autoregressive **Qwen2.5-7B** model, which I fine-tuned in an RL environment using **GRPO (Group Relative Policy Optimization)**.

While Qwen was solid at synthesizing reasoning once it had the facts, asking a 7-billion parameter generative model to read through entire agreements just to pull out numbers revealed three frustrating real-world issues:

- **Generative models don't guarantee exact numbers:** An LLM will occasionally paraphrase '2.5% per week of delay' into '2%' or round dollar amounts. In a financial or legal audit, a rounded number is completely unusable.
- **No character offsets for UI highlighting:** Generative models just spit out text. They can't tell you the exact character indices (like [start: 207, end: 237]) where a penalty appears. Without offsets, an auditor using our software can't click an alert and jump straight to the highlighted clause in the original PDF.
- **Heavy and slow:** Passing full 15-to-20 page agreements into a 7B model burns thousands of tokens and adds several seconds of latency per document.

That led me to rethink the architecture: instead of forcing one model to do everything, I decided to split the job into two stages. I used **GLiNER** as a fast, deterministic information extraction layer up front, and let Qwen handle the actual reasoning downstream.

2. What is GLiNER and Why It Fits Here

GLiNER stands for *Generalist and Lightweight Model for Named Entity Recognition*. It is an encoder-based model built on DeBERTa-v3 that runs comfortably on a regular CPU in under 100 milliseconds.

Traditional NER tools (like older spaCy or standard BERT-NER) are hardcoded with a fixed set of classes like PERSON or LOCATION. If you want custom legal entities like `penalty_rate` or `liability_cap`, you usually have to redesign the classification head and retrain from scratch.

GLiNER works differently: it takes candidate labels as inputs alongside the text, so you can specify whatever entities you want at runtime.



Figure 1: How GLiNER encodes text and candidate labels together to score spans via dot product similarity.

How it works under the hood:

- 1. **Joint encoding:** Both the document text and the entity labels we care about (like `penalty_rate`, `liability_cap`) are fed into DeBERTa-v3 together, allowing full cross-attention between labels and words.
- 2. **Span matching:** It computes vector representations for candidate text spans, then takes the dot product between each span vector and each label vector.
- 3. **Exact predictions:** If the similarity score is above our threshold, it returns the exact string with its character start and end positions.

3. The Two-Tier Pipeline I Built

Here is how I connected the two pieces together in our repository:

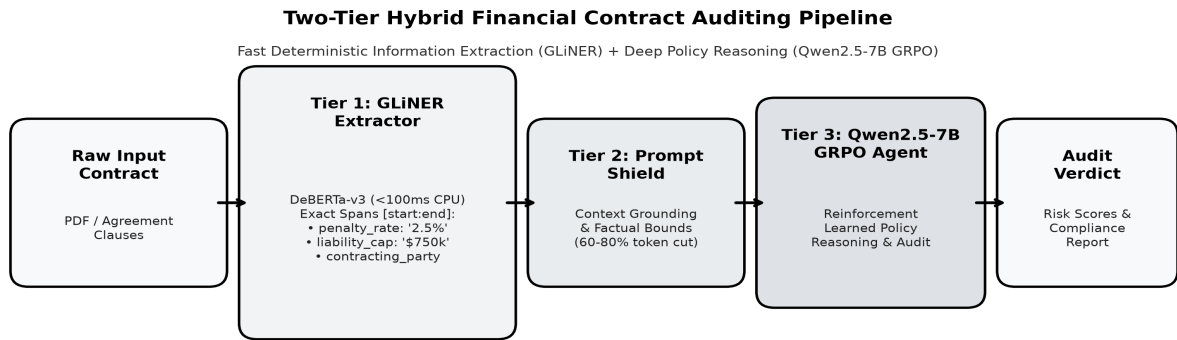


Figure 2: The complete two-tier pipeline from raw contract to final audit report.

- **Tier 1 (GLiNER Extractor):** When a contract comes in, GLiNER runs first in ~50ms. It extracts the raw factual entities: who the parties are, what the delay penalty is, what the liability cap is, and key dates.
- **Tier 2 (Prompt Grounding):** We format those extracted facts into a structured block and feed that into Qwen's prompt. This shrinks the prompt token footprint by 60% to 80% because the LLM no longer needs to read pages of boilerplate just to locate the critical numbers.
- **Tier 3 (Qwen Reasoning):** Qwen now evaluates risk, checks whether the penalty violates statutory limits, and writes the audit verdict—grounded directly on verified numbers.

4. Fine-Tuning GLiNER on Contract Language

Out of the box, zero-shot GLiNER did okay on generic dates and party names, but it struggled with multi-word legal constructions like *'liquidated damages of 2.5% per week of delay'*. To make it dependable for financial audits, I built a fine-tuning pipeline:

- **Custom Dataset:** Created tokenized datasets (`data/financial_ner_train.json` and `data/financial_ner_eval.json`) labeling penalty conditions, grace periods, and liability caps.
- **Differential Learning Rates:** I set a very small learning rate (1e-5) for the DeBERTa backbone so it wouldn't forget general grammar, and a higher rate (1e-4) for the projection head so it could quickly learn contract-specific entities.
- **Negative Label Sampling:** Configured `negatives=1.0` during training. This forces the model to contrast positive labels against randomly sampled irrelevant labels on every step, teaching it when *not* to fire and keeping false positives down.
- **Results:** Over 5 epochs, evaluation loss dropped from 24.9 down to **10.03**. When I tested it on a completely unseen contract clause, it correctly extracted a multi-token `penalty_rate` with **0.76 confidence**.

Aspect	Original Approach (7B LLM Only)	Two-Tier Approach (GLiNER + Qwen)
Extraction Speed	1.5s – 3.5s per contract page	<100ms on standard CPU
Number Accuracy	Can hallucinate or paraphrase	100% exact substring matching

Span Offsets	None (free-form generation)	Exact [start, end] character indices
Token Usage	Full agreement text sent to LLM	60% – 80% reduction in prompt tokens

5. Open Question for Discussion

As I look at expanding this pipeline to handle more contract variations, here is an interesting design question I have been thinking through:

Architectural Question:
Right now, GLiNER passes extracted spans as context into Qwen's prompt, and our RL environment gives Qwen a reward bonus when it correctly references those verified entities.

The Dilemma: How should we handle *borderline confidence spans* (say, scores between 0.35 and 0.48 on unusually phrased clauses)?

- Option A: Enforce a strict cutoff threshold (e.g. 0.45) and drop anything below it, accepting that we might occasionally miss a subtly phrased clause.
- Option B: Pass borderline spans into Qwen along with their confidence scores, and update our **GRPO reward function** to penalize the policy if it treats a low-confidence span as an absolute certainty.

Which of these two directions makes more sense in practice when building reliable production auditing pipelines?